

Security Audit

XMAQUINA (DePIN)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Audit Findings	13
Audit Remediation Summary	33
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The XMAQUINA team partnered with Hashlock to conduct a security audit of their XMQPresale.sol, XMQPresaleFactory.sol, XMQUtils.sol, XMQWhitelistNFT.sol smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

XMAQUINA is a decentralized autonomous organization (DAO) on a mission to democratize the ownership and governance of robotics, ensuring these technological advancements serve humanity as a collective.

Rooted in transparency, decentralization, and ethical innovation, the DAO is uniting a global community to collaboratively fund, develop, and share in the value of robotics and AI-driven solutions through open collaboration.

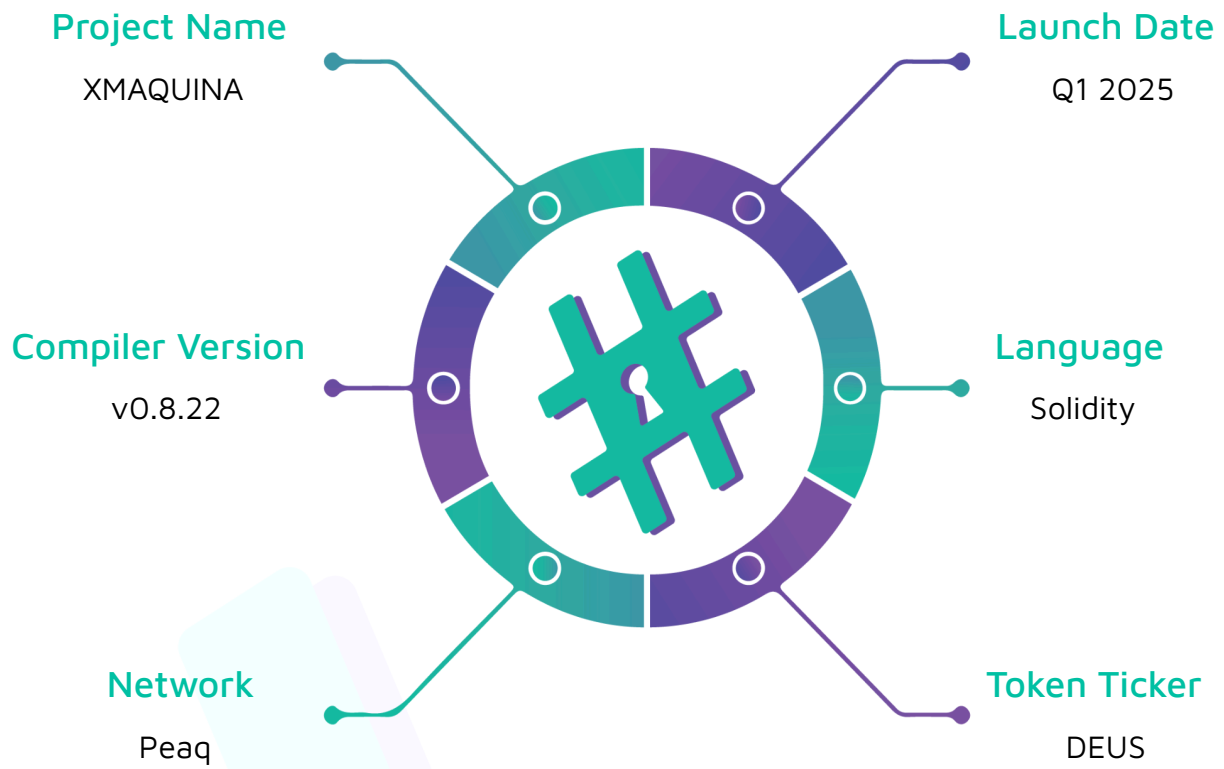
Project Name: XMAQUINA

Compiler Version: 0.8.22

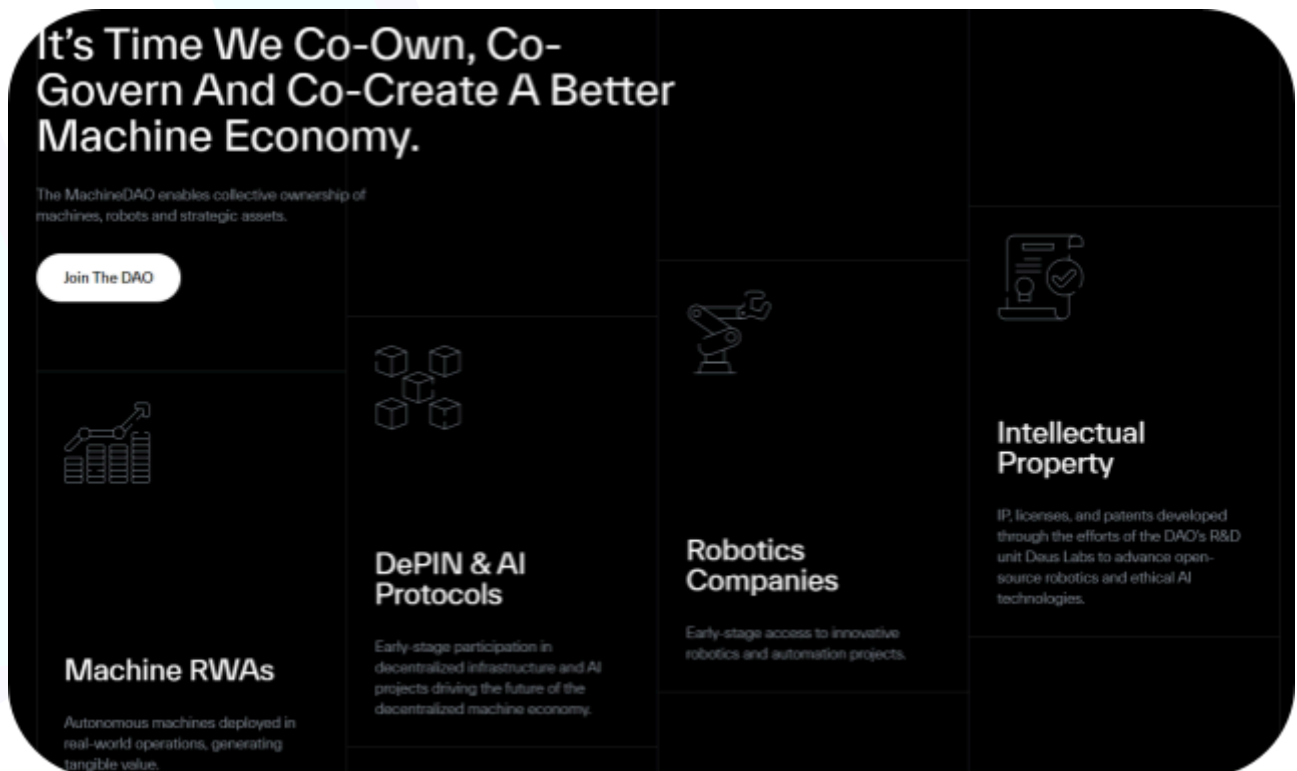
Website: www.xmaquina.io

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the XMAQUINA project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Note: The XMaquina team has provided an update to the codebase in their latest GitHub repository. After conducting a review of the changes, we confirm that the new repository contains only a few updates to the code associated with the previous contracts. These updates do not significantly alter the overall content or functionality of the codebase, and our findings remain consistent with the results outlined in the previous report, a new GitHub Commit has been renovated in the table below.

Description	XMAQUINA Protocol Smart Contracts
Platform	Peaq / Solidity
Audit Date	September, 2024
Contract 1	XMQPresale.sol
Contract 1 MD5 Hash	360fb085e86a8d580d05b651a1877b25
Contract 2	XMQPresaleFactory.sol
Contract 2 MD5 Hash	0fcdabbc5a0e0e3a203e3d39ec251d6a
Contract 3	XMQUtils.sol
Contract 3 MD5 Hash	6b4669763b9c7509a49b2d3972e1e226
Contract 4	XMQWhitelistNFT.sol
Contract 4 MD5 Hash	d9f62ba8062b5fd0d1c7a8b707ba6be5
Original GitHub Commit	516ed7bb1e1f023d42ce0181a15cb3542e640ed4
GitHub Commit v2	3a5141f2e77796aa776a2cb846db535420fb13bc
Newest GitHub Commit	baf783aacd32ffb5066d4386a0bb7ccff8cae7a5

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerabilities

3 Medium-severity vulnerabilities

6 Low-severity vulnerabilities

7 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
XMQPresale.sol - Token presale contract - Allows users to: - Purchase tokens in the presale - Claim their purchased tokens - Stake their purchased tokens - Claim a refund if the presale has failed - Allows admins to: - Withdraw funds after the presale - Pause/unpause the contract - Update due time of the presale - Update the maximum purchase amount by a single user - Update the reward protocol address	Contract achieves this functionality.
XMQPresaleFactory.sol - Contract that creates and manages individual presale contracts - Allows admin to: - Create a new presale - Update utils address	Contract achieves this functionality.
XMQUtils.sol - Utility contract - Manages whitelist, fee addresses, payment token - Allows users to: - Mint an NFT if they are whitelisted - Allows admin to:	Contract achieves this functionality.

- Set many state variables - Add/remove a user from the whitelist - Burn any user's NFT - Reset minting status of a user	
XMQWhitelistNFT.sol - Soulbound NFT contract for whitelist verification - Allows admin to: - Set the utils address - Allows the utils contract to: - Mint a new NFT - Burn any NFT	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the XMAQUINA project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the XMAQUINA project smart contract code in the form of Zip file.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] XMQUtils.sol::mintNFT - Excess Native Token amount is not refunded back to the user and this excess amount will be stuck in the contract

Description

In order for a user to mint the NFT to join the presale, they must call the `mintNFT` function in the `XMQUtils.sol` contract. This contract transfers the minting cost to the treasury address but does not consider scenarios where users might send more native tokens than this cost. The excess amount that is sent to the contract will forever be stuck.

Vulnerability Details

The `mintNFT` function shown below mints a whitelist NFT to the user calling it.

```
function mintNFT() payable external {  
    if (address(whitelistNFT) == address(0)) revert Errors.WhitelistNFTZeroAddr();  
    if (!isWhitelisted[msg.sender]) revert Errors.NotWhitelisted();  
    if (hasMintedNFT[msg.sender]) revert Errors.AlreadyMinted();  
  
    if (paymentToken == NATIVE_TOKEN_ADDRESS) {  
        if (msg.value < mintingCost) revert Errors.InsufficientFunds();  
        // Transfer the native token (ETH) to the contract's treasury  
        (bool sent, ) = treasuryAddress.call{value: mintingCost}("");  
        if (!sent) revert Errors.EtherTransferFailed();  
    } else {  
        IERC20(paymentToken).transferFrom(msg.sender, treasuryAddress,  
mintingCost);  
    }  
  
    hasMintedNFT[msg.sender] = true;
```

```

    XMQWhitelistNFT(whitelistNFT).mint(msg.sender);

    emit NFTMinted(msg.sender);
}

```

As seen from the highlighted parts, if the `paymentToken` is the Native Token, the function checks if the `msg.value` amount is equal or greater than the `mintingCost`, function will send the `mintingCost` amount to the treasury address. In a scenario where a user has sent a greater `msg.value` than the `mintingCost`, excess native tokens will not be refunded to the user. Since the contract lacks any function to withdraw native tokens in the contract, these excess tokens will forever be stuck in the contract.

Impact

Users that end up paying more than the `mintingCost` will not be refunded the excess amount. This excess amount will forever be stuck in the contract.

Recommendation

Implement refund of excess amounts in the function. An example is shown below.

```

function mintNFT() payable external {
    if (address(whitelistNFT) == address(0)) revert Errors.WhitelistNFTZeroAddr();
    if (!isWhitelisted[msg.sender]) revert Errors.NotWhitelisted();
    if (hasMintedNFT[msg.sender]) revert Errors.AlreadyMinted();

    if (paymentToken == NATIVE_TOKEN_ADDRESS) {
        if (msg.value < mintingCost) revert Errors.InsufficientFunds();

        // Transfer the native token (ETH) to the contract's treasury
        (bool sent, ) = treasuryAddress.call{value: mintingCost}("");
        if (!sent) revert Errors.EtherTransferFailed();

        // Refund excess msg.value
        uint256 excess = msg.value - mintingCost;
        if (excess > 0) {
            (bool refundSent, ) = msg.sender.call{value: excess}("");
            if (!refundSent) revert Errors.RefundFailed();
        }
    }
}

```

```
    }  
  
    } else {  
        IERC20(paymentToken).transferFrom(msg.sender, treasuryAddress, mintingCost);  
    }  
  
    hasMintedNFT[msg.sender] = true;  
    XMQWhitelistNFT(whitelistNFT).mint(msg.sender);  
    emit NFTMinted(msg.sender);  
}
```

Alternatively, make this function `nonReentrant` and implement a new function to withdraw the native tokens in the contract.

Status

Resolved

Medium

[M-01] XMQPresale.sol::purchaseTokens - It will be impossible to reach `hardCap` in certain scenarios

Description

The `purchaseTokens` function allows users to purchase tokens in the presale. The purchase is only possible when the sold amount will not reach the `hardCap`, the amount to purchase is greater than the `minPurchaseAmount`, and the user purchase will not pass the `maxPurchaseAmount` threshold. However, there can be scenarios where it will be impossible to reach the `hardCap`. This will make it so the full intended amount of tokens can not be sold.

Vulnerability Details

The `purchaseTokens` function allows users to purchase tokens in the presale as long as some requirements are met.

```
function purchaseTokens(uint256 amount) external payable nonReentrant whenNotPaused
{
    if (block.timestamp < startTime || block.timestamp > dueTime) revert Errors.InvalidTime("Presale is not active");

    if (whitelistNFT.balanceOf(msg.sender) == 0) revert Errors.NotWhitelisted();
    if (totalTokensSold + amount > hardCap) revert Errors.ExceedsHardCap();
    if (amount < minPurchaseAmount) revert Errors.BelowMinimumPurchase();

    if (userPurchases[msg.sender] + amount > maxPurchaseAmount) revert Errors.ExceedsMaxPurchase();

    //rest of the function
}
```

Imagine the following scenario where `hardCap` is 1000 tokens, `minPurchaseAmount` is 50 tokens and so far 951 tokens have been sold. The remaining 49 tokens to reach the `hardCap` will not be possible as any calls will either fail with the `ExceedsHardCap` error or the `BelowMinimumPurchase` error. Take a look at the `canClaim` function below:

```
function canClaim() public view returns (bool) {
    return (softCapReached && (totalTokensSold >= hardCap || block.timestamp > dueTime));
}
```

```
}
```

It is observed that even if a presale is successful and it should reach the **hardCap** with enough user interest in the presale since it will not be possible to reach it, all users will have to wait until **dueTime** is reached to claim their tokens.

Impact

It will be impossible to reach the **hardCap**, even if there's enough interest in wanting to purchase tokens some amount of tokens will be unpurchasable. Users will need to wait until **dueTime** is reached to claim their tokens in all situations. The unpurchasable amount of tokens will be stuck in the contract.

Recommendation

Update the function so that the full amount of tokens can be sold. If the **hardCap** - **totalTokensSold** amount is less than the **minPurchaseAmount** amount, allow the purchase of these remaining tokens.

Status

Resolved

[M-02] XMQUtils.sol::mintNFT - Function does not check for the return value of **transferFrom** which can result in users minting NFTs without paying

Description

The **mintNFT** function allows users to mint the whitelist NFT in order to join the token presale. The payment will either be in the native tokens or in an ERC20 token that is set by the protocol. In cases where a ERC20 that does not revert on transfer failure is set as the payment token users will mint NFTs without transferring any tokens.

Vulnerability Details

The **mintNFT** function shown below allows users to purchase a whitelist NFT.

```
function mintNFT() payable external {
    if (address(whitelistNFT) == address(0)) revert Errors.WhitelistNFTZeroAddr();
```

```

    if (!isWhitelisted[msg.sender]) revert Errors.NotWhitelisted();
    if (hasMintedNFT[msg.sender]) revert Errors.AlreadyMinted();

    if (paymentToken == NATIVE_TOKEN_ADDRESS) {
        if (msg.value < mintingCost) revert Errors.InsufficientFunds();
        // Transfer the native token (ETH) to the contract's treasury
        (bool sent, ) = treasuryAddress.call{value: mintingCost}("");
        if (!sent) revert Errors.EtherTransferFailed();
    } else {
        IERC20(paymentToken).transferFrom(msg.sender, treasuryAddress,
mintingCost);
    }

    hasMintedNFT[msg.sender] = true;
    XMQWhitelistNFT(whitelistNFT).mint(msg.sender);
    emit NFTMinted(msg.sender);
}

```

As observed in the highlighted part, the function does not check for the returned boolean from the `transferFrom` function. Any payment token that does not revert on failure and instead returns a boolean, this function will continue as if payment has been made even if the token transfer has failed. This will lead to users minting NFTs without transferring any tokens to the contract.

Impact

Users will be able to mint NFTs without transferring any tokens.

Recommendation

Check for the returned boolean from the `transferFrom` function, and revert if the returned boolean is false.

Status

Resolved

[M-03] XMQPresale.sol::refund - Function does not correctly update state variables

Description

When a presale fails and any user refunds their tokens with the `refund` function, this function does not update the `totalTokensSold` and `totalFundsRaised` state variables.

Vulnerability Details

When a presale fails users will call the `refund` function shown below to get their purchase refunded. This function will update the user's payments and purchases to 0 while transferring the amount they paid back to them.

```
function refund() external nonReentrant {
    if (softCapReached) revert Errors.RefundsNotAvailable();
    if (block.timestamp <= dueTime) revert Errors.PresaleStillActive();

    uint256 amount = userPayments[msg.sender];
    if (amount == 0) revert Errors.NoRefundAvailable();

    userPayments[msg.sender] = 0;
    userPurchases[msg.sender] = 0;

    if (address(paymentToken) == NATIVE_TOKEN_ADDRESS) {
        payable(msg.sender).transfer(amount);
    } else {
        paymentToken.safeTransfer(msg.sender, amount);
    }
}
```

However, taking a look into the `purchaseTokens` function which allows users to purchase presale tokens:

```
function purchaseTokens(uint256 amount) external payable nonReentrant whenNotPaused
{
    //rest of the function
}
```

```

totalTokensSold += amount;
totalFundsRaised += cost;

userPurchases[msg.sender] += amount;
userPayments[msg.sender] += cost;

//rest of the function

```

It is observed that the highlighted state variables are also updated when a purchase happens. These two state variables do not get updated when a user refunds.

Impact

State variables will have incorrect values. This can be especially problematic if any external contract or off-chain systems depend on these variables to have correct values.

Recommendation

Update the function as shown below:

```

function refund() external nonReentrant {
    if (softCapReached) revert Errors.RefundsNotAvailable();
    if (block.timestamp <= dueTime) revert Errors.PresaleStillActive();

    uint256 amount = userPayments[msg.sender];
    if (amount == 0) revert Errors.NoRefundAvailable();

    totalTokensSold -= userPurchases[msg.sender];
    totalFundsRaised -= userPayments[msg.sender];

    userPayments[msg.sender] = 0;
    userPurchases[msg.sender] = 0;

    if (address(paymentToken) == NATIVE_TOKEN_ADDRESS) {
        payable(msg.sender).transfer(amount);
    } else {
        paymentToken.safeTransfer(msg.sender, amount);
    }
}

```

Note:

The XMAQUINA team stated that the logic is functioning as intended rather than being an issue. They don't reset "totalTokensSold" and "totalFundsRaised" to zero when a presale fails because they want to maintain a record of how much was raised up until the due date. This is important for tracking purposes.

Status

Acknowledged

Low

[L-01] XMQPresale.sol - Upgradeable contract is missing __gap[..] storage variable

Description

Storage gaps are needed to not break storage layouts when adding new variables to base contracts. Listed contracts may not be inherited from but it is good practice to add it now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include __gap[..] storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited from, it is a good practice to add __gap[..] storage variables now to ensure a robust and future-proof upgradeable contract design.

Status

Resolved

[L-02] XMQPresale.sol - Missing checks for address(0) in initializer

Description

It is good practice to implement address(0) checks in the initializer or the createPresale function in the XMQPresaleFactory.sol contract to prevent potential mistakes. Following instances need address(0) checks to be implemented.

```
saleToken = IERC20(_saleToken);  
paymentToken = IERC20(_paymentToken);  
utils = XMQUtils(_utilsAddr);
```

Recommendation

Implement address(0) checks.

Status

Resolved

[L-03] XMQPresale.sol - Use of transfer or send instead of call to send native assets

Description

The use of `transfer()` in the contracts may have unintended outcomes on the native asset being sent to the receiver. The transaction will fail when:

- The receiver address is a smart contract that does not implement a payable function.
- The receiver address is a smart contract that implements a payable fallback function which uses more than 2300 gas units.
- The receiver address is a smart contract that implements a payable fallback function that needs less than 2300 gas units but is called through proxy, raising the call's gas usage above 2300.
- Additionally, using a gas value higher than 2300 might be mandatory for some multi-signature wallets.

Instances shown below:

```
line 215: payable(msg.sender).transfer(msg.value - cost);  
line 279: payable(teneoFeeAddress).transfer(teneoFee);  
line 280: payable(xmqFeeAddress).transfer(xmqFee);  
line 281: payable(treasury).transfer(treasuryAmount);  
line 302: payable(msg.sender).transfer(amount);
```

Recommendation

To transfer, a native asset is recommended to either use:

- `<receiver>.call.value(amount)`
- `OpenZeppelin Address.sendValue()`

In both cases, care must be taken not to create reentrancy vulnerabilities. Consider using [ReentrancyGuard](#) or the [Check-Effects-Interactions](#) pattern.

Status

Resolved

[L-04] XMQPresale.sol::claimAndStakeTokens - The function should [approve\(0\)](#) first

Description

Some tokens (like USDT) do not work when changing the allowance from an existing non-zero allowance value. They must first be approved by zero and then the actual allowance must be approved.

```
function claimAndStakeTokens() external nonReentrant {  
    if (rewardProtocol == address(0)) revert Errors.StakingNotAvailable();  
    if (!canClaim()) revert Errors.TokensNotClaimable();  
    uint256 amount = userPurchases[msg.sender];  
    if (amount == 0) revert Errors.NoTokensToClaim();  
  
    userPurchases[msg.sender] = 0;  
    saleToken.approve(rewardProtocol, amount);  
    IRewardProtocol(rewardProtocol).stakeOnBehalfOf(amount, msg.sender);  
}
```

Recommendation

If tokens that will be used by the protocol are known to have this behaviour, update the function to [approve\(0\)](#) before attempting to approve the amount.

Status

Resolved

[L-05] XMQUtils.sol::isEligibleForPresale - Function returns misleading boolean

Description

The function will return misleading booleans as users that have not minted but are whitelisted will cause this function to return `true` while they can't take part in the presale before they mint the NFT.

```
function isEligibleForPresale(address account) external view returns (bool) {
    return (isWhitelisted[account] && !hasMintedNFT[account]) ||
    XMQWhitelistNFT(whitelistNFT).balanceOf(account) > 0;
}
```

Recommendation

Update the function as shown below, additionally, implement another view function that will return true if an account is whitelisted and has not minted yet.

```
function isEligibleForPresale(address account) external view returns (bool) {
    return (XMQWhitelistNFT(whitelistNFT).balanceOf(account) > 0);
}
```

Status

Resolved

[L-06] Contracts - Centralization risk for privileged functions

Description

Contracts with privileged functions need the privileged role to be trusted not to perform malicious updates or drain funds. This may also cause a single point failure.

Functions shown below update critical state variables:

```
function updateUtilsAddress() //@XMQPresaleFactory.sol
function setUtils() //@XMQWhitelistNFT.sol
function pause() //@XMQPresale.sol
```

```
function unpause() //@XMQPresale.sol
function updateDueTime() //@XMQPresale.sol
function updateMaxPurchaseAmount() //@XMQPresale.sol
function updateRewardProtocol() //@XMQPresale.sol
Setter functions in //@XMQUtils.sol
function addToWhitelist() //@XMQUtils.sol
function resetMintingStatus() //@XMQUtils.sol
```

Functions shown below can be used to directly harm users, including burning a users NFT:

```
function removeFromWhitelist() //@XMQUtils.sol
function burnNFT() //@XMQUtils.sol
```

Impact

Centralization creates a single point of failure. Compromised or malicious privileged roles can harm users and the protocol.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders. This approach enhances security, reduces the risk of malicious actions by a single entity, and prevents single points of failure.

Note

XMAQUINA will utilize smart contract systems such as SAFE within good opsec recommendations to ensure the protocol remains secure for users to use.

Status

Resolved

Gas

[G-01] Contracts - Prevent setting a state variable with the same value

Description

Not only is wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances shown below:

```
function updateDueTime() // @XMQPresale.sol
function updateMaxPurchaseAmount() // @XMQPresale.sol
function updateRewardProtocol() // @XMQPresale.sol
function updateUtilsAddress() // @XMQPresaleFactory.sol
function setWhitelistNFT() // @XMQUtils.sol
function setPresaleFactory() // @XMQUtils.sol
function setTreasuryAddress() // @XMQUtils.sol
function setTeneoFeeAddress() // @XMQUtils.sol
function setXMQFeeAddress() // @XMQUtils.sol
function setPaymentToken() // @XMQUtils.sol
function setMintingCost() // @XMQUtils.sol
function setUtils() // @XMQWhitelistNFT.sol
```

Recommendation

Implement checks in your contracts to avoid setting state variables to their existing values. Prior to updating a state variable, compare the new value with the current value and proceed with the assignment only if they differ. An example shown below:

```
function setUtils(address _utils) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_utils == address(0) || _utils == utils) revert
    Errors.InvalidUtilsAddress();
    utils = _utils;
    _grantRole(MINTER_BURNER_ROLE, _utils);
}
```

Status

Resolved

[G-02] XMQPresale.sol, XMQPresaleFactory.sol - `address(this)` should be cached

Description

Caching saves gas when compared to repeating the calculation at each point it is used in the contract.

Recommendation

To enhance gas efficiency, cache the contract's address by storing `address(this)` in a state variable at the point of contract deployment or initialization. Use this cached address throughout the contract instead of repeatedly calling `address(this)`. This practice reduces the gas cost associated with multiple computations of the contract's address, leading to more efficient contract execution, especially in scenarios with frequent usage of the contract's address.

Status

Acknowledged

[G-03] XMQPresale.sol::purchaseTokens - `msg.value` should be cached

Description

Caching saves gas when compared to repeating the calculation at each point it is used in the contract.

Recommendation

Consider caching `msg.value` in a local variable at the start of payable functions where it is accessed multiple times. Implement this optimization by assigning `msg.value` to a local variable and using this variable for subsequent operations within the function. An example shown below:

```

function purchaseTokens(uint256 amount) external payable nonReentrant whenNotPaused
{
    //rest of the function

    uint256 wholeTokens = amount / (10 **
IERC20Metadata(address(saleToken)).decimals());

    uint256 cost = wholeTokens * tokenPrice;
    uint256 cachedValue = msg.value;

    if (address(paymentToken) == NATIVE_TOKEN_ADDRESS) {
        if (cachedValue < cost) revert Errors.InsufficientPayment();
        if (cachedValue > cost) {
            payable(msg.sender).transfer(cachedValue - cost);
        }
    }

    //rest of the function

```

Status

Resolved

[G-04] XMQPresale.sol::event PresaleEnded - Event declared but not emitted

Description

An event within the contract is declared but not utilised in any of the contract's functions or operations. Having unused event declarations can consume unnecessary space and may lead to misunderstandings for developers or users expecting this event as part of the contract's functionality.

Recommendation

Consider removing the declared `PresaleEnded` event that is not being emitted.

Status

Resolved

[G-05] XMQPresale.sol, XMQUtls.sol - State variables that are used multiple times in a function should be cached

Description

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it. Instances shown below:

```
@XMQPresale.sol

line 256: saleToken.approve(rewardProtocol, amount); // State variable
`rewardProtocol` is used also on line(s): ['250', '257'].

line 281: payable(treasury).transfer(treasuryAmount); // State variable `treasury` is
used also on line(s): ['285'].

line 324: if (newDueTime <= dueTime) revert Errors.InvalidTime("New due time must be
after current due time"); // State variable `dueTime` is used also on line(s):
['325'].

@XMQUtls.sol

line 217: XMQWhitelistNFT(whitelistNFT).mint(msg.sender); // State variable
`whitelistNFT` is used also on line(s): ['203'].

line 213: IERC20(paymentToken).transferFrom(msg.sender, treasuryAddress, mintingCost);
// State variable `paymentToken` is used also on line(s): ['207'].

line 213: IERC20(paymentToken).transferFrom(msg.sender, treasuryAddress, mintingCost);
// State variable `mintingCost` is used also on line(s): ['208', '210'].
```

Recommendation

Cache state variables in stack or local memory variables within functions when they are used multiple times. Example shown below:

```
function deposit(address _payer) external onlyRole(FACTORY) {
    uint256 cachedHardCap = hardCap;

    if (saleToken.balanceOf(address(this)) > 0) revert
    Errors.DepositAlreadyMade();

    saleToken.safeTransferFrom(_payer, address(this), cachedHardCap);

    if (saleToken.balanceOf(address(this)) < cachedHardCap) revert
    Errors.InsufficientFunds();
}
```

Status

Resolved

[G-06] XMQPresaleFactory.sol::implementation - State variables only set in the constructor should be declared immutable

Description

Avoids a `Gsset(20000 gas)` in the constructor, and replaces the first access in each transaction(`Gcoldload - 2100 gas`) and each access thereafter(`Gwarmacces - 100 gas`) with a `PUSH32( 3 gas )`.

Recommendation

Consider marking state variables as an immutable that never changes on the contract.

Status

Acknowledged

[G-07] XMQWhitelistNFT.sol::mint - `x += y` costs less gas than `x = x + y` for state variables

Description

The compound assignment `x += y` has been proven to consume less gas compared to `x = x + y` when updating state variables. This approach is not only gas-efficient but also aligns with best coding practices, offering cleaner, more readable code.

Recommendation

Update the function. An example is shown below:

```
function mint(address to) external onlyRole(MINTER_BURNER_ROLE) {
    uint256 newTokenId = _tokenIds;
    _tokenIds += 1;
    _safeMint(to, newTokenId);
}
```

Status

Resolved

Audit Remediation Summary

XMaquina Smart Contract Audit Report – Findings Remediation and Updates

This section outlines the resolution of issues identified in the audit conducted. The findings were checked by Hashlock and the differences are stated below. The fixes address vulnerabilities, enhance flexibility, and align the code with best practices, ensuring the smart contracts meet high security and functionality standards.

XMQ and Teneo Fees – From Static to Dynamic

- **Audit Finding:** Centralization risk for privileged functions and inefficiencies in static fee handling.
 - **Resolution:**
 - Introduced dynamic setter functions allowing the owner to update **XMQ** and **Teneo** fees during operation.
 - Implemented checks to prevent setting fees to the same values unnecessarily, mitigating gas wastage (as per Gas Optimization G-01).
 - Emission of specific events (**FeesUpdated**) to track fee changes, enhancing transparency and monitoring.
 - **Comparison to Findings:** Centralized control over critical functions was noted as a potential risk. The implementation of setter functions combined with safe operational practices aligns with audit recommendations for robust owner control and transparency.
-

Dynamic Whitelist NFT Functionality

- **Audit Finding:** The whitelist logic in **isEligibleForPresale** returned misleading booleans, limiting flexibility.
- **Resolution:**

- Introduced a setter function to dynamically manage the whitelist NFT requirements for presale participation.
 - Added an event (**WhitelistRequirementsUpdated**) to notify changes to the whitelist logic.
 - Enhanced **isEligibleForPresale** to correctly reflect whether an account is eligible based on both whitelist status and minted NFT status, avoiding the misleading results noted in L-05.
 - **Comparison to Findings:** This resolves the earlier limitation where whitelist eligibility wasn't dynamically manageable and ensures presale eligibility is accurately determined.
-

New Event Emissions

- **Audit Finding:** The lack of emitted events for key updates reduced contract transparency.
 - **Resolution:**
 - Events such as **FeesUpdated** and **WhitelistRequirementsUpdated** are emitted whenever corresponding setter functions are invoked.
 - Removed unused or redundant events (e.g., **PresaleEnded**) as per Gas Optimization G-04.
 - **Comparison to Findings:** This change addresses transparency concerns by ensuring all significant updates are logged on-chain and aligns with best practices for event management.
-

Updated Comments for Enhanced Clarity

- **Audit Finding:** Comments were well-aligned but required updates for new functionality.
- **Resolution:**

- Comprehensive updates were made to comments and NatSpec documentation to reflect the newly implemented dynamic functions and their purpose.
 - **Comparison to Findings:** Maintaining updated comments ensures continued code clarity and aids in future audits or modifications.
-

Summary of Resolved Audit Issues

- **High Severity:** The vulnerability in `mintNFT` regarding excess native tokens was resolved by refunding excess amounts or allowing withdrawals.
 - **Medium Severity:** Issues such as hard cap inefficiencies and unchecked transfer return values were addressed by updating purchase logic and adding appropriate checks.
 - **Low Severity & Gas Optimizations:** All identified inefficiencies (e.g., storage gaps, unused variables, and redundant operations) were fixed to ensure efficient and secure contract execution.
-

The XMaquina smart contracts now incorporate robust mechanisms to enhance security, transparency, and flexibility, successfully addressing all findings from the audit.

Centralisation

The XMAQUINA project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the XMAQUINA project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#Hashlock.

#Hashlock.

Hashlock Pty Ltd